# Python Module

# Why Python Modules?

## ❌ Without Modules

```
# all_my_code.py  (500+ lines!)
def greet(): ...
def add(): ...
def draw_chart(): ...
```

- Hard to find anything
- Hard to share functions
- Easy to break by accident
- No teamwork possible

## ✅ With Modules

```
# greetings.py  |  math_tools.py
# charts.py     |  each < 100 lines
```

- Easy to navigate
- Reuse across projects
- Test independently
- Collaborate with a team

# What is Python Moduel?

A module is simply a **.py file** you can import.

📁 *Think of a module like a toolbox — each file holds related tools, and you only open the box you need, when you need it.*

### 🏛️ Built-in

Ships with Python. No install needed.

*os, math, random, datetime...*

### 📦 Third-party

Made by the community. Install with pip.

*requests, numpy, pandas...*

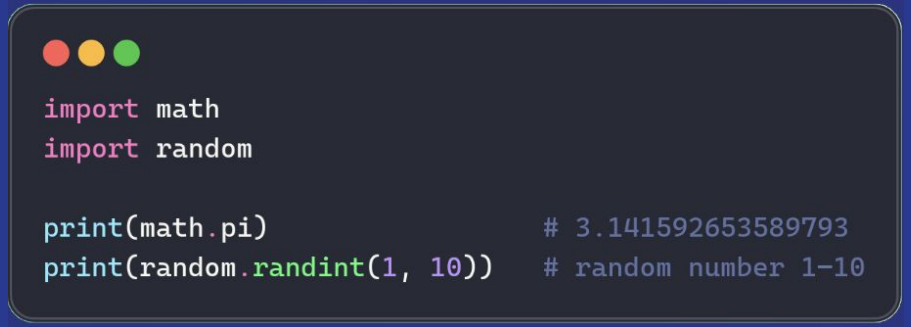
### ✍️ Your own

Files you write yourself. Save as .py and import!

*my_tools.py, utils.py...*

# Using Python Modules

1. Type import, then the module name — no quotes needed.
2. Access anything inside using module.something (dot notation).
3. Python searches your project folder then its standard library



SYNTAX

# Using Alias and from...import

## from … import

Import only what you need — keeps the namespace clean.

```
from math import sqrt, pi

print(sqrt(16))   # 4.0
print(pi)         # 3.14...
```

**Insight:** avoid from math import * — it imports everything and can silently overwrite your own variable names.

## import … as (alias)

Give a module a shorter nickname — very common in data science.

```
import numpy as np
import pandas as pd

np.array([1, 2, 3])
```

# Python's built-in modules covers most everyday tasks

### 💻 os

File & folder operations

*os.listdir('.')*

### 🔢 math

Math functions & constants

*math.sqrt(144)*

### 🎲 random

Random numbers & choices

*random.choice(list)*

### 📅 datetime

Dates and times

*datetime.date.today()*

### 📄 json

Read/write JSON data

*json.loads(text)*

### ⚙️ sys

Python interpreter info

*sys.argv[1]*

# 500,000+ Packages via pip install

```
$ pip install requests
Successfully installed requests-2.31.0
```

### requests

HTTP calls — fetch web pages & APIs

### numpy

Fast number crunching & arrays

### pandas

Spreadsheet-style data analysis

### pillow

Open, edit & save images

# Creating your own module

### STEP 1 — Create greetings.py

```
# greetings.py
def say_hello(name):
    return f'Hello, {name}!'

def say_bye(name):
    return f'Bye, {name}!'
```

### STEP 2 — Import it in main.py

```
# main.py  (same folder)
import greetings

print(greetings.say_hello('Alice'))
print(greetings.say_bye('Bob'))
```

▶ **Output:** Hello, Alice!  Bye, Bob!

# Prevent code from running when import

- When you run a file directly, Python sets **__name__** to "__main__".
- When it's imported, __name__ is the module's file name instead.

```python
# greetings.py
def say_hello(name):
    return f'Hello, {name}!'

if __name__ == '__main__':
    # runs ONLY when this file is executed directly
    print(say_hello('Test User'))
```

✅ **run python greetings.py** → prints 'Hello, Test User!'

✅ **import greetings** elsewhere → nothing prints automatically

# Practice exercises

🛠️ TRY IT YOURSELF

# Build a Calculator Module in 5 Minutes

📄 **calc.py — your module**

```python
# calc.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

# TODO: add multiply & divide
```

📄 **main.py — use your module**

```python
# main.py
import calc

print(calc.add(10, 5))        # 15
print(calc.subtract(10, 5))   # 5

# Challenge: add multiply!
```

**Insight:** 🎯 Challenge: add a multiply(a, b) and divide(a, b) function to calc.py.